Dt:- 31/07/26

# UNIT-II :-

## Dynamic Programming / Greedy Algorithms

Module I  Module II

### (*) Dynamic Programming :- (2M)

- Dynamic Programming used in Optimization Problem.
- Dynamic Programming solves Problem by Combining the Solutions of Sub-problems
- It Solves each subproblem just once and then save its Answer in a table to avoid recomputing of all the steps.

### (*) Steps of Dynamic Programming Approach (2M)

1) Characterize the structure of an optimal Solution.
2) Recursively define the Value of an optimal Solution.
3) Compute the Value of an Optimal Solution, bottom-up fashion
4) Construct an optimal solution from the Computed information.

(*) Elements of Dynamic Programming (2M)

1) Sub structure :- Decompose the given Problem into the smaller subproblems

2) Table Structure :- After solving a problem store the result of sub-problems to the table-structure

3) Bottom-up-computation :- Using the table combine the solution using bottom-up approach.

| Divide & Conquer | Dynamic Programming. |
|---|---|
| 1) It deals the steps divide, conquer & combine | 1) It deals with the steps Characterize, recursive, Compute & Construct. |
| 2) It does more work on subproblems, hence computations are more | 2) It solves the subproblem only once and stole in the table. |
| 3) Sub-problems are independent of each other | 3) Sub-problems are interdependent on each other. |
| 4) Uses Top-down Approach | 4) Uses Bottom-up Approach. |
| Ex:- 5) Binary Search, Quick sort, Merge sort | 5) Rod-Cutting Problem, longest-Common problem & Matrix Multiplication. |

05/08/26

# Rod-Cutting Problem 5M/7M/8M

- optimization Problem & Solved using Dynamic Programming.

**Recurrence**

$r[0] = 0$

$r[j] = \max(\text{Price}[i] + r[j-i])$ → cut చేసిన తర్వాత rod లో మిగిలిన ముక్కలు

i → length of rod taking.

j → rod length (లేదా) cut possibilities

**Example:-**

Rod having length $n = 4$

| Length (i) | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| Price ($p_i$) | 1 | 5 | 8 | 9 |

**Solution:-**

**step 1**

$r[0] = 0$

If you have a rod length 0, you have nothing to sell

0 length = 0 profit

Hence $\boxed{r[0] = 0}$

**step 2 :-**

$r[i]$ - Rod length 1.

The rod is 1 length long, Hence Possible Cut is 1 Only (we can't cut 2, 3, 4)

1 unit length ని ఇక్కడ cut చేయలేం.

$i = 1$

$= \text{Price}[1] + r[1-1]$

$= \text{Price}[1] + r[0]$

$= 1 + 0 \Rightarrow 1$

$\boxed{r[1] = 1}$

3) Step 3: $r[2]$ – length 2

What is max Profit for 2 inch rod &

Choice $i=1$ & $i=2$ | రెండు. ముక్కలు చేయవచ్చు

If $i=1$ (cut 1 | leave 1)

$= \text{Price}[i] + r[2-i]$

$= \text{Price}[1] + r[1]$

$= 1+1 \Rightarrow 2$

$r[2] = 2$

---

If $i=2$ (cut 2 | leave 0)

$= \text{Price}[2] + r[2-2]$ → length → ఎన్ని cut చేసాం అది

$= \text{Price}[2] + r[0]$

$= 5+0$

$= 5$

$r[2] = 5$

$r_2 = \max(2,5)$

max = 5

Next స్టెప్ లో $r_2$ యొక్క Max తీసుకోవాలి.

Table for Computation

| j (rod length) | j=1 (cut ఇవ్వ) | j=2 | j=3 | j=4 | r[j] max | Best cut |
|---|---|---|---|---|---|---|
| 0 | – | – | – | – | 0 | – |
| 1 | 1+0=1 | – | – | – | 1 | 1 |
| 2 | 1+1=2 | 5+0=5 | – | – | 5 | 2 |
| 3 | 1+5=6 | 5+1=6 | 8+0=8 | – | 8 | 3 |

Dt: 06/08/26

## Algorithm

1) start
2) r[0] = 0
3) let r[0 . . . . n]
4) for j = 1 to n
5) max_Value = -∞
6) for i = 1 to j

   max_val = max (max Val, price[i] + r[j - 1]

   r[j] = max_val
7) return r[n]
8) stop

Time Complexity :- $(On^2)$
Space Complexity :- $(On)$

## Longest Common Subsequence :-

04/08/26

- The longest Common Subsequence (LCS) Problem finds the length of the longest Subsequence present in two strings in the same relation.
- A Subsequence is a string generated from the Original String by deleting 0 (or) more characters without changing the relative order of the remaining character.

$S_1$
$S_2$

$S_1$
$S_2$

Recurrence

If $S_1$

→

[

else

dp [

[if

ele

Problem

$S_1 = A$

$S_2 = G$

Solve

rows = $S_1$

1) Creat

to 0

2) set

$S_1 = ABC$ — matching
$S_2 = ACD$ } Subsequence → AC

$S_1 = ABCBDAB$
$S_2 = BDCAB$ } Subsequences.

ABCBDAB
BDCAB
Possibility 1 : BCAB
Possibility 2 : BDAB

**Recurrence Calculations :-**

If $S_1[i] = S_2[j]$

⟶ $dp[i][j] = dp[i-1][j-1] + 1$

dynamic Programming.

[diagonal value +1] if match

else

$dp[i][j] = \max(dp[i][j], dp[i][j-1])$

[if not match then Consider the greater element from diagonal elements]

**Problem** CIA

$S_1 = AGGTAB$

$S_2 = GXTXAYB$

Solve the example using longest Common Subproblem

row = $S_1$ col = $S_2$

1) Create Table $dp[m+1]$ & $[n+1]$ & initialized to 0.

2) set the first row & 1st column to 0

[+1 rows, columns we have to make]

[చివర్లో +1 row, clm లో 0 పెట్టాలి.]

[మనకు కొంచెం max తీసుకోవాలి]

[diagonal up ఉంటే equal elements]

| | ∅ | G | X | T | X | A | Y | B |
|---|---|---|---|---|---|---|---|---|
| ∅ | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| A | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 |
| G | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| G | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
| T | 0 | 1 | 1 | 2 | 2 | 2 | 2 | 2 |
| A | 0 | 1 | 1 | 2 | 2 | 3 | 3 | 3 |
| B | 0 | 1 | 1 | 2 | 2 | 3 | 3 | 4 |

match అవ్వకపోతే. horizontal గా ఉంటే అక్క పక్కన చూడాలి

(subsequence కోసం Bottom-up Approach వాడడం జరిగింది)

(Diagonal up వెళ్లాలి) → matching అయితే horizontal చూడాలి

**Steps :-** [for Exam]

→ $dp[G][G] = dp[\phi][X]+1 = 1$

→ $dp[T][T] = dp[G][X]+1$

$1+1=2$

→ $dp[A][A] = dp[T][X]+1 = 2+1=3$

→ $dp[B][B] = dp[A][Y]+1 = 3+1=4$

---

(B, B) match → B → (A, Y) → ~~left~~ up move

(A, A) match → A → (T, X) → left move

(T, T) match → T → (G, X) → up/left

(G, G) match → G → Completed.

Reversing the corrected characters order is GTAB

→ 4 length.

Date: 13/08/26

Example : (2)

$S_1$ = A B C B D A B

$S_2$ = B D E A B

L.C.S =

| | ∅ | B | D | E | A | B |
|---|---|---|---|---|---|---|
| ∅ | 0 | 0 | 0 | 0 | 0 | 0 |
| A | 0 | 0 | 0 | 0 | 1 | 1 |
| B | 0 | 1 | 1 | 1 | 1 | 2 |
| C | 0 | 1 | 1 | 1 | 1 | 2 |
| B | 0 | 1 | 1 | 1 | 1 | 2 |
| D | 0 | 1 | 2 | 2 | 2 | 2 |
| A | 0 | 1 | 2 | 2 | 3 | 3 |
| B | 0 | 1 | 2 | 2 | 3 | 4 |

B D A B → zero skips diagonally

Algorithm

1) Create a table dp[m+1][n+1], initialized to 0 → ∅ పెట్టడానికి

2) Set first row & 1st column to 0

3) for i=1 to m

for j=1 to n

If $S_1[i-1] = S_2[j-1]$

$dp[i][j] = dp[i-1][j-1]+1$

else

$dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$

dp[m][n] = length of L.C.S

5) Traceback to get L.C.S

- start at i=m, j=n
  while i>0 & j>0
- If $s_1[i-1] = s_2[j-1]$ add character to result, move diagonally.
- else if $dp[i-1][j] > dp[i][j-1]$
  move - up left
- Reverse the string.

## (*) Module - II (*)

## Greedy Algorithm

- When a Problem has many feasible solutions with different cost (or) benefit, finding the best Solution is known as an optimization Problem & the best Soln is known as Optimal Solution.

→ Greedy Method Constructs the Solution Simple by looking at Current benefit without exploring the future possibilities & hence they are known as "Greedy"

### Types of Greedy Solutions

1) feasible :- Choice should satisfy the Problem Constraint

2) Locally optimal :- Best Solutions from all feasible Solution at Current stage.
(Dynamic → Globally optimal)

4) Irrevocable :- Once choice is made it can't be altered.

(x) Applications of Greedy Approach :-

1) knapsack Problem
2) Minimum Spanning Tree
3) Single Source Shortest Path
4) Activity Selection Problem
5) Job Sequencing Problem.

Difference b/w Greedy Method & Dynamic Programming (5M/7M/2M)

| Dynamic Programming | Greedy Method. |
|---|---|
| 1) DP guarantees an optimal soln | 1) Greedy Approach doesn't guarantees optimal soln |
| 2) Subproblems can overlap | 2) Subproblems do not overlap |
| 3) DP considers future-choices | 3) It considers only the current choices |
| 4) DP select the choice which is Globally Optimum | 4) It selects the choice which is locally optimum |
| 5) DP contains Memoization | 5) Greedy Approach donot contain Memoization. |
| 6) App's :- Rod-cutting Problem, Longest common Subsequences etc | App's :- knapsack Pblm, Activity selection pblm, Min. Spanning Tree, Job-sequencing Pblm. |

Dt: 14/08/26

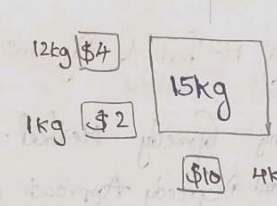
(11) Knap Sack Problem : Fractional knapsack.

→ knapsack is a kind of a bag with limited weight capacity

→ The value (or) Profit obtained by putting the items into the knapsack is maximum

→ And the weight limit of the knapsack doesn't exceed.

→ In fractional knapsack items are divisible.




2 types
- Normal knap ఒక్కటే చెప్పు లేదు
- Fractional ఒక్కటే మొత్తం item తీసుకోవచ్చు

Example: For the given set of items & knapsack capacity = 60kg, find the optimal soln for the fractional knapsack pblm using greedy Approach (Greedy Method) (Best soln)

$n = 5$

$(w_1, w_2, w_3, w_4, w_5) = (5, 10, 15, 22, 25)$

$(B_1, B_2, B_3, B_4, B_5) = (30, 40, 45, 77, 90)$

Solution

Compute the $\frac{\text{value}}{\text{weight}}$ ratio for each item.

| Items | Weight | Value | Ratio |
|---|---|---|---|
| 1 | 5 | 30 | 6 |
| 2 | 10 | 40 | 4 |
| 3 | 15 | 45 | 3 |
| 4 | 22 | 77 | 3.5 |
| 5 | 25 | 90 | 3.6 |

Step 2 :- Sort all the items in decreasing order of their Value/weight ratio.

| $I_1$ | $I_2$ | $I_3$ | $I_4$ | $I_5$ |
|---|---|---|---|---|
| 6 | 4 | 3.6 | 3.5 | 3 |

Step 3 :- Start filling the knapsack by putting items one by one.

| Knapsack weight | Items in Knapsack | Cost |
|---|---|---|
| 60 | ∅ | 0 → ఏమీ fill చెయ్యలేదు |
| 55 (60-5) ← remaining capacity | $I_1$ (wt 5) | 30 → value |
| 45 (55-10) | $I_1$, $I_2$ (next లో $I_2$ add చేశాం) | 30+40=70 |
| 20 | $I_1$, $I_2$, $I_5$ | 70+90 = 160 |

(పైన వచ్చిన ratio order చేసుకోవాలని చెప్పాం కదా)

highest order లో sequence లో $I_4$ కి 22 capacity లేదు కాబట్టి 20 వే వేసుకోండి.

20 కోసం 20 వేసుకోండి.

Now the knapsack weight left to fill is 20kg but item 4 has weight 22 kg

— In ~~fractional~~ knapsack we can do the fraction of items.

2 wt కి value 77

20 కి ఎంత profit ⇒ $\frac{20}{22} \times 77$

→ Items will be (lot remaining, Actual unit of item)

$\langle I_1, I_2, I_5, (20/22) \times I_4 \rangle$

$= 160 + (20/22) \times 77$ (value)

$= 160 + 70$

$= 230$ Units

(Hence the items Put in the knapsack are $I_1, I_2, I_5$ & $(20/22)\ I_4$ with total cost 230 units).

(*) Activity Selection Problem → non-overlapping Activities we have to identify.

| Activity | start Time | finish Time |
|---|---|---|
| $A_1$ | 1 | 3 |
| $A_2$ | 2 | 5 |
| $A_3$ | 4 | 7 |
| $A_4$ | 6 | 9 |
| $A_5$ | 8 | 10 |
| $A_6$ | 9 | 11 |

Note :- Arrange the Activities Accoding to their finish Time. (Min-Max finish Time)

Step 1 :- Select $A_1$

Start = 1 finish = 3 $A_1$ అయ్యేవరకు $A_2$ start అవ్వలేము.

Step 2 :- check $A_2$

start = 2

$A_2$ start $< 2 < 3$ ($A_1$ finish Time) → Reject

start 2 finish time ఉంటేది.

Step 3 :- Check $A_3$

Start = 4 $A_1$ :- 3 → Finish.

$4 \geq 3 \rightarrow$ Select $A_3$

finish = 7 ↳ ఇక్కడ finish time next కి తీసుకోవాలి

Step 4 :- check $A_4$

Start 6

$6 < 7 \rightarrow$ Reject

Step 5 :- Check $A_5$

Start = 8

$8 \geq 7$

Select $A_5$

finish = 10

Step 6 :- Check $A_6$

Start = $9 < 10 \rightarrow$ Reject

Final Selection :-

The selected Activities are

$A_1, A_3, A_5$

∴ The Maximum no. of non-Overlapping Activities = 3

Algorithm (chatgpt)

Algorithm ActivitySelection (S, F, n)

1) select the first Activity.
2) Print Activity 1.
3) set i = 1
4) for j = 2 to n do

```
If S[j] >= F[i] then
    Print Activity j
    i=j
  End If
End fo
```

5) stop

Dt:- 27/08/26

## Activity Selection Problem Algorithm

Activity_selection (start[], finish[], n]

1) Sort all Activities According to increasing finish time
2) Select the first Activity
3) Set last_finish = finish[0]
4) fo i=1 to n-1;
   If start[i] ≥ last_finish
   Select activity i
   last_finish = finish[i]
5) Stop

### Complexities

Time C. O(n) } If sorted.
Space C. O(1) }

Time C. O(n log n) } If Not sorted.
Space C. O(n) }

## Huffman Coding :- (Greedy)

Huffman coding is a lossless data compression technique that assigns variable length binary code to characters based on how often they occur

Example :- B C A A C

| character | ASCII value | Binary |
|---|---|---|
| A | 65 | 01000001 → 8 bits |
| B | 66 | 01000010 |
| C | 67 | 01000011 |

Total length = 5

5 × 8 (bits) = 40

If Length = 15 then 15 × 8 = 120

So, we can not create 40 bits, but we can create our own coding system using Huffman code to ~~Compare~~ Compress the data

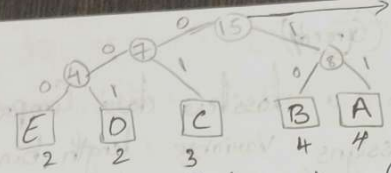
Example :-

B C A A C B D E A C E D B A B

Characters = 15

Sol$^n$ :-

| Character | Count | Code | Bits × Counts (→ Total msg value) |
|---|---|---|---|
| A | 4 | 11 | 2 × 4 = 8 |
| B | 4 | 10 | 2 × 4 = 8 |
| C | 3 | 01 | 2 × 3 = 6 |
| D | 2 | 001 | 3 × 2 = 6 |
| E | 2 | 000 | 3 × 2 = 6 |
| 5 characters | 15 Count | 12 bits | 34 bits |




→ Huffman root

[అయితే Minimum వ్యాల్యూస్ రావాలి, ఆ రావ్యాలకి Next Min → అలా మొదటికి రావాల్సి correct]

[Tree కి left branches కి 0
right branches కి 1]

Now while Sending the Compressed message decoding table also needs to be Sent.

Hence, we need to Send the Unique identification Number while decoding.

5 × 8 bit ASCII characters

= 40 bit → 5 characters & value.

& + code bits

Hence 40 bits + 12 bit = 52 bits

↳ మరియు compress చేసిన value

Message → 34 bits

↳ Total Msg value.

Table for encoding → 52 bits

34 + 52 bits = 86 bits

మొత్తం characters ← 15 × 8 = 120

120 bits > 86 bits → మనం Compress చేసినట్టే.

x bits

Hence Huffman Coding Works for lossless data Compression.

Dt: 02/09/26

## Huffman Coding Algorithm

1) Create a leaf node for each character,

Weight = frequency → small to big frequencies రావాలి.

2) Insert all leaf nodes into a min-Priority queue, ordered by frequency.

3) While more than One node remains in the queue

- Remove the two nodes with Smallest frequency & call them x & y
- Create a new internal node z.

freq(z) = freq(x) + freq(y)

- Make x left child & y right child of z.
- Insert 'z' back to the priority queue

4) The One node left in the queue is the root of the Huffman tree

5) Traverse the tree from root to each leaf:
label edges left : 0
right : 1

The bit string created from root to leaf node is the Huffman code.

## Fibonacci Heap ÷ Not Greedy method. Part of Advanced data structure.

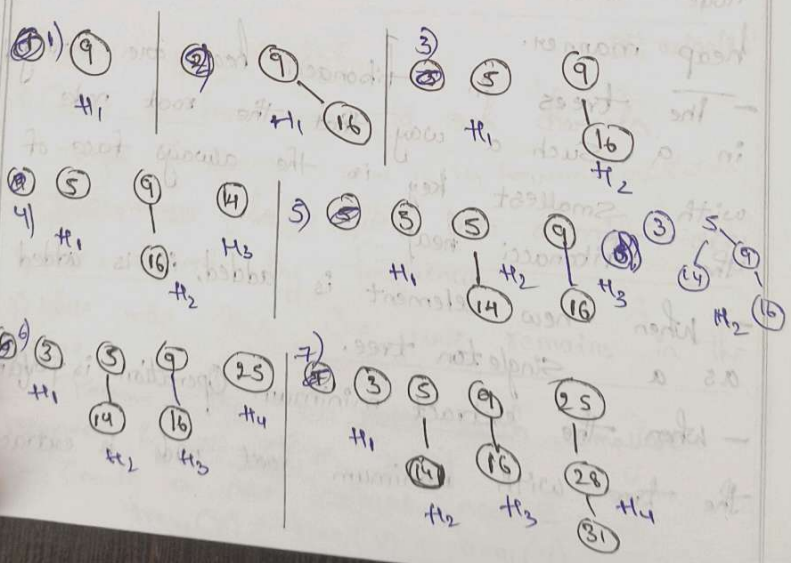
- It is a data structure used for implementing Priority queue
- A fibonacci heap is a collection of trees where each tree is a heap-ordered multi-tree meaning that each tree has a single-root node with its children arranged in a heap manner.
- The trees in fibonacci heap are arranged in a such a way that the root node with smallest key is the always face of the fibonacci heap → Min is always smallest value.
- When new element is added, it is added as a singleton tree.
- When the extract minimum operation is performed, the tree with minimum root node is extracted.

- fibonacci heap maintain Pointer Value & all roots are Connected Using doubly circular linked lists.

Example :-

9, 16, 5, 14, 3, 25, 28, 31

| Insert | Root list after insert | Min. |
|---|---|---|
| 9 | 9 | 9 |
| 16 | 9, 16 | 9 |
| 05 | 9, 16, 05 | 9 |
| 14 | 9, 16, 05, 14 | 05 |
| 3 | 9, 16, 05, 14, 3 | 3 |
| 25 | 9, 16, 05, 14, 3, 25 | 3 |
| 28 | 9, 16, 05, 14, 3, 25, 28 | 3 |
| 31 | 9, 16, 05, 14, 3, 25, 28, 31 | 3 |



final

